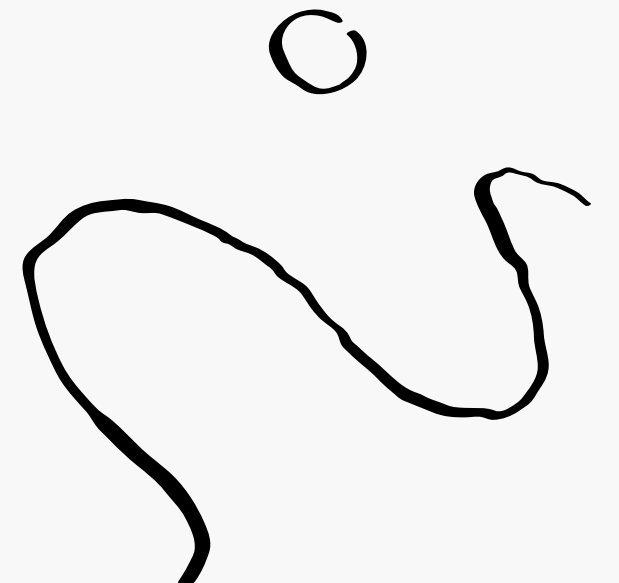
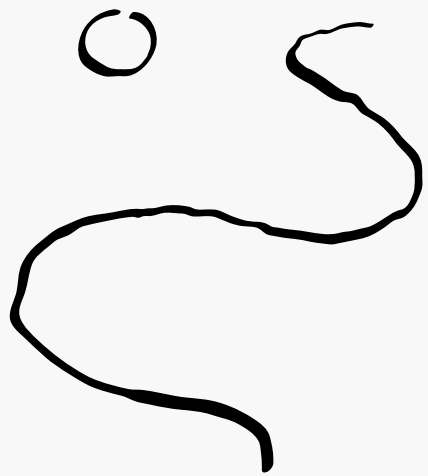
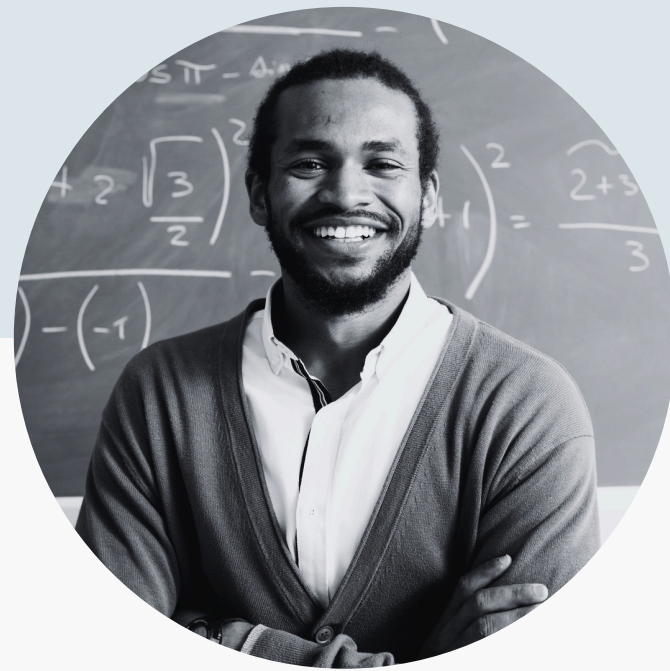




Web chuyển đổi ảnh thành tranh vẽ



Thành viên nhóm



Ma Văn Hùng

B22DCCN363



Bùi Văn Hiến

B22DCCN292



Nội dung chính

1. **Giới thiệu đề tài**
2. **Nền tảng lý thuyết và triển khai**
3. **Kết quả đạt được**

Giới thiệu đề tài



-
- Xây dựng một ứng dụng web để tự động chuyển đổi ảnh chụp thành tranh vẽ bút chì với chất lượng cao, giao diện thân thiện, và hiệu năng tốt.
 - Chức năng
 - Cho phép người dùng upload ảnh từ máy tính
 - Chuyển đổi ảnh sang sketch với kết quả chất lượng cao Hỗ trợ tùy chỉnh các tham số xử lý (phương pháp, cường độ)
 - Cho phép tải xuống kết quả
 -



Nền tảng lý thuyết và triển khai

Pipeline chuyển ảnh thành tranh vẽ bút chì

INPUT: RGB Image ($H \times W \times 3$)

↓ Grayscale Conversion

Gray Image ($H \times W$) [Weighted Average]

↓ Preprocessing (Smoothing)

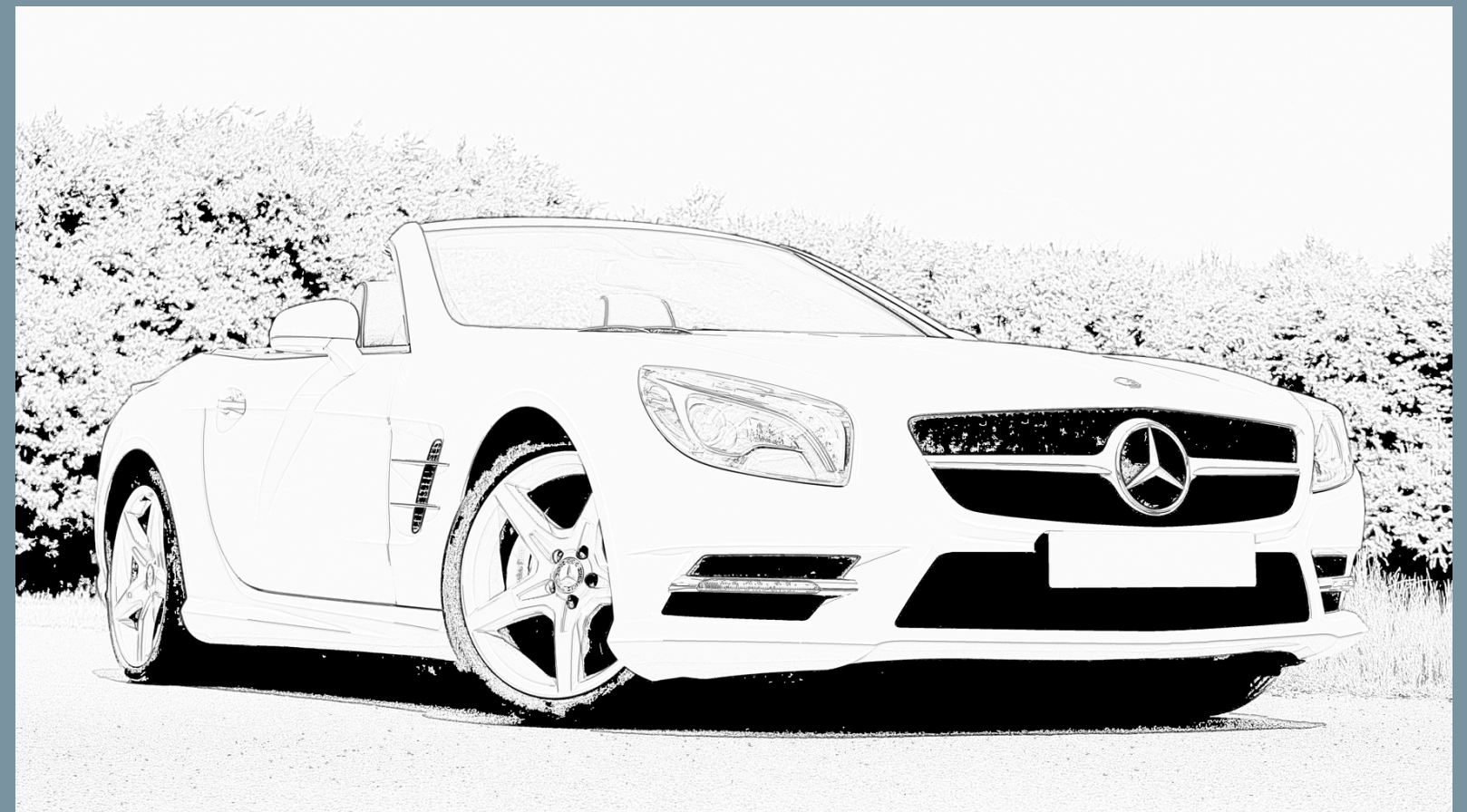
Smoothed Image (Bilateral/Gaussian/Median)

↓ Edge Detection

Edge Map (Canny/Sobel/Laplacian)

↓ Sketch Effect (Dodge Blending)

OUTPUT: Final Sketch ($H \times W$, grayscale)



Chuyển ảnh thành ảnh xám



Ảnh RGB (Red, Green, Blue) là ảnh màu với 3 kênh độc lập, mỗi kênh có giá trị từ 0-255. Khi chuyển sang ảnh xám (grayscale), ta giảm xuống 1 kênh duy nhất đại diện cho độ sáng (luminosity).

Phương pháp - Weighted Average (Luminosity Method)

Công thức:

$$\text{Gray}(x, y) = 0.299 \times R(x, y) + 0.587 \times G(x, y) + 0.114 \times B(x, y)$$

- 0.299 (Red): Mắt người nhạy cảm ít nhất với ánh sáng đỏ → hệ số nhỏ nhất
- 0.587 (Green): Mắt người nhạy cảm nhất với ánh sáng xanh lá → hệ số lớn nhất (gần 59%)
- 0.114 (Blue): Mắt người nhạy cảm ít nhất với ánh sáng xanh dương → hệ số nhỏ nhất



Chuyển ảnh thành ảnh xám



```
import numpy as np
def convert_to_grayscale(image):
    if len(image.shape) == 2:
        return image.astype(np.uint8)

    image = image.astype(np.float32)

    if len(image.shape) == 3 and image.shape[2] == 3:
        r = image[:, :, 0]
        g = image[:, :, 1]
        b = image[:, :, 2]
        gray = 0.299 * r + 0.587 * g + 0.114 * b
        return np.clip(gray, 0, 255).astype(np.uint8)

    if len(image.shape) == 3 and image.shape[2] == 4:
        rgb = image[:, :, :3]
        gray = 0.299 * rgb[:, :, 0] + 0.587 * rgb[:, :, 1] + 0.114 * rgb[:, :, 2]
        return np.clip(gray, 0, 255).astype(np.uint8)

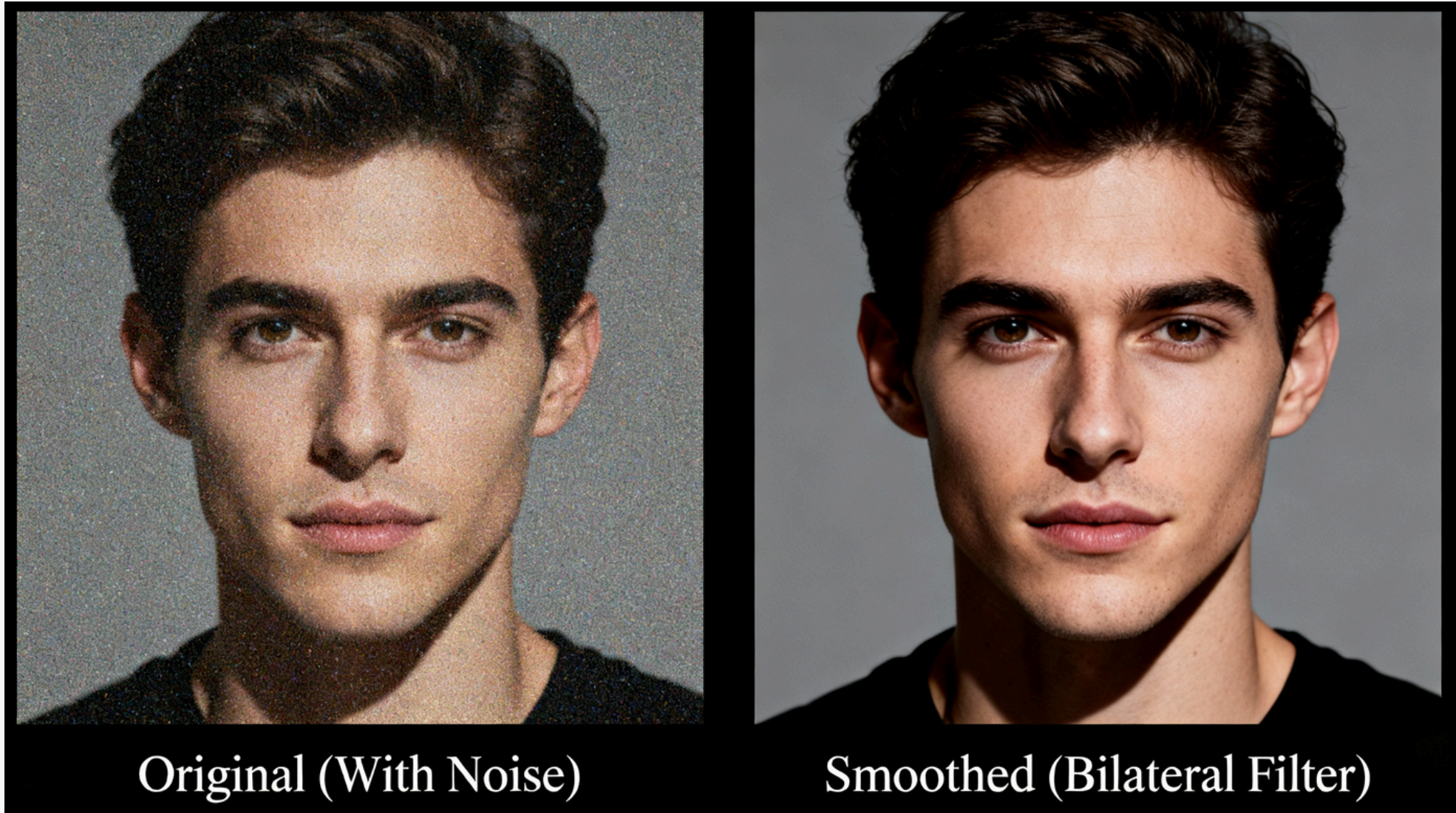
    return image.astype(np.uint8)
```



Làm mịn ảnh

Mục đích

- Giảm nhiễu: Loại bỏ noise (nhiều camera, nhiều sensor)
- Tăng tính ổn định: Chuẩn bị cho phát hiện biên
- Làm đơn giản: Giảm bớt chi tiết không quan trọng



Làm mịn ảnh

Gaussian Blur- Làm mịn đều

Sử dụng phân phối Gauss (Gaussian distribution) để tính weighted average của các pixel lân cận. Pixel ở gần tâm có trọng số cao, pixel ở xa có trọng số thấp.

Hàm Gaussian 2 chiều

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} \cdot \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

Trong đó:

- x,y: khoảng cách từ pixel xét đến tâm kernel
- σ : độ lệch chuẩn (điều khiển mức độ làm mờ)
- Giá trị $G(x,y)$ chính là trọng số của pixel tại vị trí (x,y)

```
def apply_gaussian_blur(image, kernel_size=(5, 5)):
    """Gaussian Blur - Sử dụng scipy.ndimage"""
    ksize_h = kernel_size[0] if kernel_size[0] % 2 == 1 else kernel_size[0] + 1
    ksize_w = kernel_size[1] if kernel_size[1] % 2 == 1 else kernel_size[1] + 1

    # Tính sigma từ kernel size
    sigma = max(ksize_h, ksize_w) / 6.0

    # Sử dụng scipy.ndimage.gaussian_filter (tối ưu C backend)
    return ndimage.gaussian_filter(image.astype(np.float64), sigma=sigma).astype(np.uint8)
```

Làm mịn ảnh

Median Filter

Thay thế mỗi pixel bằng giá trị median (trung vị) của tất cả pixel trong cửa sổ lân cận

Quy trình:

- Sắp xếp các pixel trong cửa sổ theo thứ tự tăng dần: $p_1 \leq p_2 \leq \dots \leq p_n$
- 2. Lấy giá trị ở vị trí giữa:
$$\text{median} = p_{(n+1)/2}$$

```
def apply_median_blur(image, kernel_size=5):  
    """Median Blur - Sử dụng scipy.ndimage"""  
    ksize = kernel_size if kernel_size % 2 == 1 else kernel_size + 1  
  
    # Sử dụng scipy.ndimage.median_filter  
    return ndimage.median_filter(image, size=ksize).astype(np.uint8)
```

Làm mịn ảnh

```
def apply_bilateral_filter_manual(image, d=9, sigma_color=75, sigma_space=75):  
    radius = d // 2  
    height, width = image.shape[:2]  
  
    # Pre-compute spatial Gaussian weights  
    y_coords, x_coords = np.ogrid[-radius:radius+1, -radius:radius+1]  
    spatial_weights = np.exp(-(x_coords**2 + y_coords**2) / (2 * sigma_space**2))  
  
    # Pad image  
    padded = np.pad(image.astype(np.float64), radius, mode='reflect')  
    filtered = np.zeros_like(image, dtype=np.float64)  
  
    image_float = image.astype(np.float64)  
    # Áp dụng filter trên từng pixel  
    for i in range(height):  
        for j in range(width):  
            neighborhood = padded[i:i+d, j:j+d]  
            center_value = image_float[i, j]  
  
            # Compute color weights  
            color_diff = neighborhood - center_value  
            color_weights = np.exp(-(color_diff**2) / (2 * sigma_color**2))  
  
            # Combined weights  
            weights = spatial_weights * color_weights  
            weights_sum = np.sum(weights)  
  
            # Weighted average  
            if weights_sum > 0:  
                filtered[i, j] = np.sum(weights * neighborhood) / weights_sum  
            else:  
                filtered[i, j] = center_value  
  
    result = np.clip(filtered, 0, 255).astype(np.uint8)
```

Bilateral Filter

Kết hợp 2 Gaussian kernels để vừa làm mịn vừa bảo tồn biên:

- Spatial Gaussian: Dựa trên khoảng cách giữa pixel (proximity)
- Range Gaussian: Dựa trên độ khác biệt giá trị pixel (similarity)

$$BF(x) = \frac{1}{W_p} \sum_i [G_s(\|x - x_i\|) \cdot G_r(\|I(x) - I(x_i)\|) \cdot I(x_i)]$$

Trong đó:

- G_s : Gaussian spatial, tính từ khoảng cách Euclidean
- G_r : Gaussian range, tính từ độ khác giá trị
- W_p : Hệ số chuẩn hóa (tổng các trọng số)
- σ_{space} , σ_{color} : Độ lệch chuẩn cho spatial & range

Phát hiện biên

- Biên (Edge) là những vị trí có sự thay đổi đột ngột về cường độ sáng / giá trị pixel.

Phát hiện biên là bước quan trọng trong xử lý ảnh:

- Làm đơn giản ảnh bằng cách giữ lại thông tin quan trọng nhất
 - Giúp nhận diện đối tượng (object detection)
 - Nền tảng cho sketch effect
-

Sobel Edge Detection

Laplacian Edge Detection

Canny Edge Detection

Sobel

Sử dụng hai kernel convolution để tính gradient (đạo hàm bậc 1) theo hướng x và y.

Sobel X:

Gx = [-1 0 1]
[-2 0 2]
[-1 0 1]

Sobel Y:

Gy = [-1 -2 1]
[0 0 0]
[1 2 1]

Quy trình:

1. Tính gradient X: $G_x = \text{Sobel_X} * \text{Image}$ (convolution)
2. Tính gradient Y: $G_y = \text{Sobel_Y} * \text{Image}$ (convolution)
3. Tính magnitude: $M = \sqrt{G_x^2 + G_y^2}$

```
def sobel_edge_detection(image):  
    # Sobel kernels  
    sobel_x = np.array([[ -1,  0,  1],  
                        [ -2,  0,  2],  
                        [ -1,  0,  1]], dtype=np.float64)  
  
    sobel_y = np.array([[ -1, -2, -1],  
                        [  0,  0,  0],  
                        [  1,  2,  1]], dtype=np.float64)  
  
    # Apply convolution  
    image_float = image.astype(np.float64)  
    grad_x = ndimage.convolve(image_float, sobel_x)  
    grad_y = ndimage.convolve(image_float, sobel_y)  
  
    # Compute magnitude  
    magnitude = np.sqrt(grad_x**2 + grad_y**2)  
    magnitude = np.clip(magnitude, 0, 255)  
    return magnitude.astype(np.uint8)
```

Laplacian

Sử dụng đạo hàm bậc 2 (Laplacian operator) để phát hiện edges tại vị trí zero-crossing (nơi giá trị từ dương sang âm hoặc ngược lại).

Kernel Laplacian:

$$L = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$$

Quy trình

1. Tính Laplacian: $L = \text{Kernel} * \text{Image}$
2. Lấy absolute value: $|L|$
3. Threshold hoặc zero-crossing detection

```
def laplacian_edge_detection(image):  
    # Laplacian kernel  
    laplacian_kernel = np.array([[0, 1, 0],  
                                [1, -4, 1],  
                                [0, 1, 0]], dtype=np.float64)  
  
    # Apply convolution  
    image_float = image.astype(np.float64)  
    laplacian = ndimage.convolve(image_float, laplacian_kernel)  
    laplacian = np.abs(laplacian)  
    laplacian = np.clip(laplacian, 0, 255)  
    return laplacian.astype(np.uint8)
```


Canny

```
def canny_edge_detection(image, low_threshold=50, high_threshold=150):
    rows, cols = image.shape[:2]

    # 1. Gaussian smoothing
    gaussian_kernel = np.array([[1, 2, 1],
                                [2, 4, 2],
                                [1, 2, 1]], dtype=np.float64) / 16
    smoothed = ndimage.convolve(image.astype(np.float64), gaussian_kernel)

    # 2. Sobel gradient
    sobel_x = np.array([[ -1,  0,  1],
                        [ -2,  0,  2],
                        [ -1,  0,  1]], dtype=np.float64)
    sobel_y = np.array([[ -1, -2, -1],
                        [  0,  0,  0],
                        [  1,  2,  1]], dtype=np.float64)

    grad_x = ndimage.convolve(smoothed, sobel_x)
    grad_y = ndimage.convolve(smoothed, sobel_y)
    magnitude = np.sqrt(grad_x**2 + grad_y**2)

    # Tính góc gradient (radian → degree), chuẩn hóa về [0, 180)
    angle = np.arctan2(grad_y, grad_x) * 180.0 / np.pi
    angle[angle < 0] += 180
```

Bước 1: Gaussian Smoothing

Bước 2: Gradient Calculation

Bước 3: Non-Maximum Suppression

Bước 4: Double Threshold & Hysteresis

```

1
2 # 3. Non-Maximum Suppression (NMS)
3
4 nms = np.zeros_like(magnitude)
5
6 for i in range(1, rows - 1):
7     for j in range(1, cols - 1):
8         q, r = 255, 255
9
10        # Xác định 2 pixel lân cận theo hướng gradient
11        # 0° (ngang): so sánh trái-phải
12        if (0 <= angle[i, j] < 22.5) or (157.5 <= angle[i, j] <= 180):
13            q = magnitude[i, j + 1]
14            r = magnitude[i, j - 1]
15        # 45° (chéo): so sánh trên-phải và dưới-trái
16        elif 22.5 <= angle[i, j] < 67.5:
17            q = magnitude[i - 1, j + 1]
18            r = magnitude[i + 1, j - 1]
19        # 90° (dọc): so sánh trên-dưới
20        elif 67.5 <= angle[i, j] < 112.5:
21            q = magnitude[i - 1, j]
22            r = magnitude[i + 1, j]
23        # 135° (chéo): so sánh trên-trái và dưới-phải
24        elif 112.5 <= angle[i, j] < 157.5:
25            q = magnitude[i - 1, j - 1]
26            r = magnitude[i + 1, j + 1]
27
28        # Giữ lại nếu là cực đại cục bộ
29        if magnitude[i, j] >= q and magnitude[i, j] >= r:
30            nms[i, j] = magnitude[i, j]
31        else:
32            nms[i, j] = 0
33

```

Canny

Bước 1: Gaussian Smoothing

Bước 2: Gradient Calculation

Bước 3: Non-Maximum Suppression

Bước 4: Double Threshold & Hysteresis



```
1  # 4. Double Threshold
2  strong_val = 255
3  weak_val = 75  # giá trị tạm cho weak edges
4
5  strong_edges = nms >= high_threshold
6  weak_edges = (nms >= low_threshold) & (nms < high_threshold)
7
8  result = np.zeros_like(nms, dtype=np.uint8)
9  result[strong_edges] = strong_val
10 result[weak_edges] = weak_val
11
12 # 5. Hysteresis - Edge tracking by connectivity
13 # Weak edge chỉ được giữ nếu kết nối với strong edge
14 for i in range(1, rows - 1):
15     for j in range(1, cols - 1):
16         if result[i, j] == weak_val:
17             # Kiểm tra 8-connectivity với strong edge
18             neighborhood = result[i-1:i+2, j-1:j+2]
19             if np.any(neighborhood == strong_val):
20                 result[i, j] = strong_val
21             else:
22                 result[i, j] = 0
23
24 return result
25
26
```

Canny

Bước 1: Gaussian Smoothing

Bước 2: Gradient Calculation

Bước 3: Non-Maximum Suppression

Bước 4: Double Threshold & Hysteresis

Tạo hiệu ứng Sketch



Phương pháp Pencil Sketch

Dựa trên kỹ thuật Dodge blending trong Photoshop, tạo hiệu ứng "vẽ bút chì tay" bằng cách:

- Invert ảnh grayscale Gaussian blur ảnh inverted
- Áp dụng Dodge blending
- Kết hợp với edge map

Các bước tạo hiệu ứng nét vẽ

Bước 1: Invert image

Bước 2: Gaussian Blur

Bước 3: Dodge Blending

$$S = I / (255 - I_{\text{blur}} + \epsilon) \times 255$$

Trong đó:

- I : Ảnh grayscale gốc (layer dưới)
- I_{blur} : Ảnh blur inverted (layer trên)
- ϵ (epsilon): Giá trị nhỏ ($1e-7$) để tránh chia cho 0
- S : Kết quả sketch

Bước 4: Blend with edges

```
def create_pencil_sketch(gray_image, edges):  
    # Invert image  
    inverted = 255 - gray_image  
  
    # Blur inverted image  
    blurred = gaussian_blur_manual(inverted, kernel_size=21)  
  
    # Dodge blending  
    def dodge(front, back):  
        result = back.astype(np.float64) / (255 - front.astype(np.float64) + 1e-7) * 255  
        result = np.clip(result, 0, 255)  
        return result.astype(np.uint8)  
  
    sketch = dodge(blurred, gray_image)  
  
    # Blend with edges  
    edges_inv = 255 - edges  
    sketch = (sketch.astype(np.float64) / 255.0 *  
              edges_inv.astype(np.float64) / 255.0 * 255).astype(np.uint8)  
  
    return sketch
```



Thank you

